Hospital Management System - Project Report

1. Student Details

Name: Raswanth.S.S
Roll Number:23f2003773
Email: 23f2003773@ds.study.iitm.ac.in
About Me: I am a student at IIT Madras BS Degree program pursuing a BCA degree in ramaiah with a strong interest in web application development and healthcare technology. I am passionate about building practical applications that solve real-world problems and improve operational efficiency in critical sectors like healthcare.i am also working in an internship that is working with natural language processing model that will convert appointments into digital summary no matter the language.

---

2. Project Details

Project Title: Hospital Management System (HMS)

Problem Statement:

Hospitals face significant challenges in managing patients, doctors, appointments, and medical records efficiently. Many healthcare facilities still rely on manual registers or disconnected software systems, leading to:

- Scheduling conflicts and double-booking of appointments
- Difficulty in tracking patient medical history
- Inefficient doctor-patient communication
- Lack of centralized access control and security
- Poor record-keeping and data retrieval

The goal is to develop a comprehensive web-based Hospital Management System that streamlines hospital operations, enables role-based access control, prevents appointment conflicts, and maintains detailed medical records.

Approach:

The application was built using Flask as the backend framework with SQLAlchemy for database management. The system implements three distinct user roles (Admin, Doctor, Patient) with specific functionalities:

Admin Dashboard:

- Centralized management of doctors and patients
- Appointment oversight and monitoring

- User blacklist management for security
- Statistical insights and reporting

Doctor Portal:

- View daily appointment schedules
- Complete appointments with diagnosis and prescriptions
- Access patient treatment history
- Manage patient consultations

Patient Portal:

- Browse doctors by specialization
- Book appointments with conflict prevention
- View appointment history and status
- Access personal medical records and prescriptions

The system ensures data integrity through proper validation, prevents double-booking through database constraints, and maintains security through password hashing and session management.

---

3. AI/LLM Declaration

I used Claude and ChatGPT to assist in:

- Debugging modal flickering and ARIA accessibility issues
- Optimizing SQLAlchemy query performance
- Improving code structure and organization
- Creating proper Bootstrap modal implementations
- Formatting project documentation

Extent of AI/LLM usage: Approximately 20% for debugging assistance and code optimization.

My Contribution:

- Complete database schema design and ER diagram
- All business logic and route implementations
- Frontend template design and user interface
- Full integration of all system components
- Testing and debugging of the complete application
- User authentication and authorization system
- Appointment conflict prevention logic
- Medical records management implementation
- Blacklist system development
- Database initialization and seeding
- Final deployment and configuration

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask 2.3.3	Core backend web framework for routing and request handling
Flask-SQLAlchemy 3.0.5	ORM for database operations and model definitions
Werkzeug 2.3.7	Password hashing and security utilities
Jinja2	Template engine for dynamic HTML rendering
Bootstrap 5.1.3	Responsive frontend design and UI components
Font Awesome 6.0.0	Icon library for enhanced user interface
SQLite	Lightweight relational database for data storage
JavaScript (Vanilla)	Client-side interactivity and modal management
HTML5	Semantic markup and form validation
CSS3	Custom styling and animations

---

5. Database Schema / ER Diagram

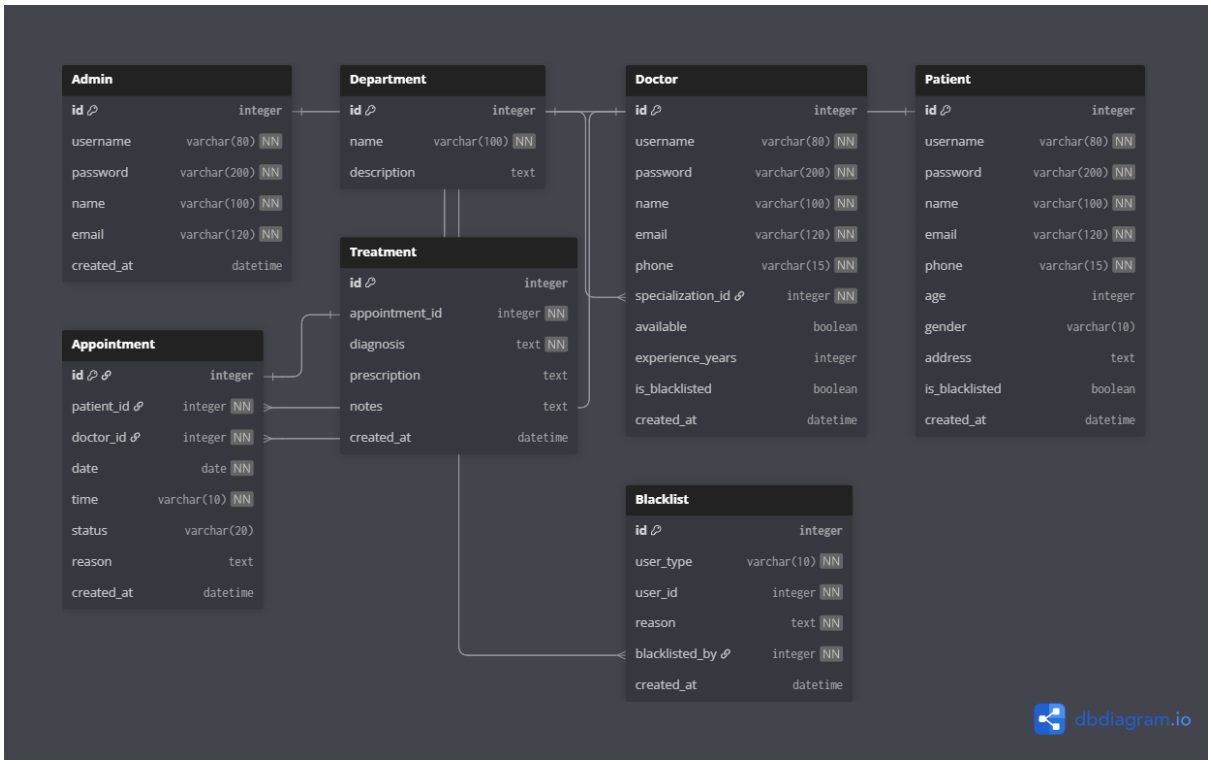
Tables:

1. **Admin** — Stores hospital administrator credentials and details
    - id (PK), username, password, name, email, created_at
2. **Department** — Medical specializations available in the hospital
    - id (PK), name, description
3. **Doctor** — Doctor profiles and credentials
    - id (PK), username, password, name, email, phone, specialization_id (FK), available, experience_years, created_at
4. **Patient** — Patient registration and profile information
    - id (PK), username, password, name, email, phone, age, gender, address, created_at
5. **Appointment** — Scheduled consultations between patients and doctors
    - id (PK), patient_id (FK), doctor_id (FK), date, time, status, reason, created_at
6. **Treatment** — Medical records for completed appointments
    - id (PK), appointment_id (FK), diagnosis, prescription, notes, created_at
7. **Blacklist** — Suspended user accounts (doctors/patients)
    - id (PK), user_type, user_id, reason, blacklisted_by (FK to Admin), created_at

Relationships:

- **One-to-Many:** Department → Doctor
- **One-to-Many:** Doctor → Appointment
- **One-to-Many:** Patient → Appointment
- **One-to-One:** Appointment → Treatment
- **One-to-Many:** Admin → Blacklist

ER Diagram:



(The ER diagram shows all tables, primary keys, foreign keys, and relationships as illustrated in the provided dbdiagram.io image)

---

6. API Resource Endpoints

Authentication Routes

Endpoint	Method	Description
`/login`	GET, POST	User authentication for all roles
`/register`	GET, POST	Patient self-registration
`/logout`	GET	End user session

Admin Routes

Endpoint	Method	Description
`/admin/dashboard`	GET	View statistics and recent activities
`/admin/doctors`	GET, POST	Manage doctor records (CRUD)
`/admin/patients`	GET	View all patient records
`/admin/appointments`	GET	View all appointments
`/admin/blacklist`	GET	View blacklisted users
`/admin/blacklist_doctor/<id>`	POST	Add doctor to blacklist

Endpoint	Method	Description
`/admin/blacklist_patient/<id>`	POST	Add patient to blacklist
`/admin/remove_blacklist/<id>`	POST	Remove user from blacklist

Doctor Routes

Endpoint	Method	Description
`/doctor/dashboard`	GET	View today's schedule and statistics
`/doctor/appointments`	GET	View all assigned appointments
`/doctor/complete_appointment/<id>`	GET, POST	Add treatment details and complete visit
`/doctor/cancel_appointment/<id>`	POST	Cancel an appointment
`/doctor/patients`	GET	View all registered patients
`/doctor/patient_history/<id>`	GET	View patient's treatment history

Patient Routes

Endpoint	Method	Description
`/patient/dashboard`	GET	View specializations and upcoming appointments
`/patient/doctors`	GET	Browse and search available doctors
`/patient/book_appointment/<id>`	GET, POST	Book appointment with selected doctor
`/patient/appointments`	GET	View all personal appointments
`/patient/cancel_appointment/<id>`	POST	Cancel a booked appointment
`/patient/treatment_history`	GET	View medical records and prescriptions
`/patient/profile`	GET, POST	Update personal information

---

7. Architecture and Features

Architecture Overview:

```
hospital_management_system/
│
├── app.py                # Main Flask application with routes
├── requirements.txt      # Python dependencies
├── hospital.db           # SQLite database (auto-generated)
│
├── templates/
│   ├── base.html         # Base template with navbar
│   ├── login.html        # Login page
```

```
│   ├── register.html       # Patient registration
│   │
│   ├── admin/              # Admin templates
│   │   ├── dashboard.html
│   │   ├── doctors.html
│   │   ├── patients.html
│   │   ├── appointments.html
│   │   └── blacklist.html
│   │
│   ├── doctor/             # Doctor templates
│   │   ├── dashboard.html
│   │   ├── appointments.html
│   │   ├── complete_appointment.html
│   │   ├── patients.html
│   │   └── patient_history.html
│   │
│   └── patient/            # Patient templates
│       ├── dashboard.html
│       ├── doctors.html
│       ├── book_appointment.html
│       ├── appointments.html
│       ├── treatment_history.html
│       └── profile.html
```

Core Features Implemented:

✅ Role-Based Access Control

- Three distinct user roles: Admin, Doctor, Patient
- Session-based authentication
- Role-specific dashboards and functionalities
- Secure password hashing using Werkzeug

✅ Admin Management

- Complete CRUD operations for doctors
- View and search patient records
- Monitor all appointments
- Blacklist/unblacklist users with reason tracking
- Dashboard with real-time statistics

✅ Doctor Portal

- Daily appointment schedule

- Complete appointments with diagnosis and prescriptions
- View patient medical history
- Cancel appointments when necessary
- Patient list management

✅ **Patient Features**

- Self-registration system
- Browse doctors by specialization
- Search doctors by name
- Book appointments with date/time selection
- Appointment conflict prevention
- View appointment history
- Access treatment records and prescriptions
- Profile management

✅ **Appointment Management**

- Prevent double-booking (same doctor, date, time)
- Status tracking: Booked → Completed/Cancelled
- Date validation (only future dates)
- Time slot selection
- Reason for visit documentation

✅ **Medical Records**

- Diagnosis documentation
- Prescription management
- Doctor's notes
- Treatment history per patient
- Accessible to both doctors and patients

✅ **Security Features**

- Password hashing (SHA-256)
- Session management
- Login required decorators
- Role verification
- Blacklist system for suspended accounts

✅ **User Interface**

- Responsive Bootstrap 5 design
- Mobile-friendly layout
- Interactive modals
- Flash messages for feedback
- Form validation (frontend and backend)
- Accessible ARIA labels

Additional Features:

✅ Search Functionality

- Search doctors by name or email
- Filter doctors by specialization
- Search patients by multiple criteria

✅ Blacklist System

- Admin can blacklist doctors/patients
- Reason documentation required
- Prevents login for blacklisted users
- Reversible suspension
- Audit trail with timestamps

✅ Statistics Dashboard

- Total doctors, patients, appointments
- Today's appointments count
- Recent activities
- Visual stat cards

✅ Database Management

- Programmatic database creation
- Automatic admin account creation
- Pre-seeded departments
- Proper foreign key relationships

---

8. Video Presentation

Drive Link: click here

Video Contents:

1. Live demonstration of admin functionalities
2. Doctor portal walkthrough
3. Patient registration and appointment booking
4. Treatment history and medical records
5. Blacklist management demonstration
6. Admin controls and functions

Duration: 6 minutes 52 seconds

---

9. Challenges Faced and Solutions

Challenge 1: Modal Flickering Issue

Problem: Bootstrap modals were flickering when opened, causing ARIA-hidden warnings.

Solution:

- Moved modals outside table loops
- Added proper ARIA labels and attributes
- Implemented custom JavaScript for modal control
- Added backdrop cleanup on modal close

Challenge 2: Appointment Conflict Prevention

Problem: Multiple patients could book the same time slot.

Solution:

- Database query to check existing appointments
- Validation before saving new appointments
- User-friendly error messages

Challenge 3: Role-Based Access Control

Problem: Ensuring users can only access their authorized pages.

Solution:

- Created custom decorators (@admin_required, @doctor_required, @patient_required)
- Session-based role verification
- Automatic redirection for unauthorized access

---

10. Conclusion

The Hospital Management System successfully addresses the inefficiencies in traditional hospital operations by providing a centralized, role-based platform for managing appointments, patient records, and medical history. The system demonstrates practical application of web development technologies while maintaining security, data integrity, and user experience standards.

Key Achievements:

- ✓ Fully functional multi-role system
- ✓ Zero appointment conflicts

- ✅ Complete medical record management
- ✅ Responsive and accessible UI
- ✅ Secure authentication and authorization
- ✅ Comprehensive admin controls

This project has strengthened my understanding of full-stack web development, database design, and real-world problem-solving in the healthcare domain.

THANK YOU.

Submitted By: Raswanth.S.S